

Tips and Tricks

Disclaimer: Please DON'T copy and paste code. Take some time to understand why its done that way and ask questions if necessary.

For anyone interested on tips in regards to writing unit test.

0- Structure of writing a unit test

1- Want to run individual test instead of all test method?

2- Don't know why test need coverage? Run report

3- How to mocking dependency using Mockery?

4- How to fake a https server for testing calls like ``wp_safe_remote_get``?

5- How to get past `wp_verify_nonce`?

6- How to unit test CMB2 stuff?

7- How to unit test `wp_json_send` / `wp_die` / die for rendering json?

8- How to mock built-in wordpress function? (sometime this works)

9- How do I test all these HTML echo / dump?

10- How to unit test `filter_input`?

11 - How to test filters?

12 - How to unit test `get_the_term`?

13 - How do I test wp-cli commands?

14- How to test ``is_admin``?

15- How to unit test `get_term`?

Tips:

0- Structure of writing a unit test

```

/**
 *      test someMethod
 *
 *      In this method, I want to test this function call generate_xml()
 *
 *      @cover HelperUtil\XML::generate_xml
 *      ^ In the cover part, I reference the class that this function is
in
 *      ^ "Cover" tells phpunit which method in HelperUtil\XML class is
this test covering
 *      ^ Best Practice: Use only 1 cover per method, too many method
makes it hard to test due to less cohesion
 */
public function test_generate_xml() {
    //      Step 1: (Optional) See if there is any global you want
<though this is bad practice to use global in a unit test>
    $global $wp_filter;

    //      Step 2: (Optional) Mock anything you need
    add_action('get_the_terms', '__return_true');

    $_POST['name'] = 'my-first-xml';

    $xmldata_mock = $this->createMock('xmldata::class');
    $xmldata_mock
                                ->expects($this->any())           //      When is
this mock apply?
                                ->method('get_xml_data')//      The method name
its mocking
                                ->with('science data')           //      If
get_xml_data has a parameter like "type" -> get_xml_data($type)
                                ->willReturn(['result' => 'my result']);

    //      Step 3: Call that function your mocking
    $xml = new HelperUtil\XML();
    $actual_result = $xml->generate_xml();

    //      Final Step: Perform the assertion
    $expected_result = 'something its expected to return'; // Whatever
the expected result is suppose to be
    $this->assertEquals($expected_result, $actual_result); // Though
prefer this code was inline and not have those 2 variable
}

```

- As your writing test, it can be a pain having to run entire test suite (over and over) when you only want to debug 1 test method. (or generate a report for 1 method)

```
./bin/phpunit --config ./circleci/phpunit.xml --filter=test_foobar
```

This allow you to run a specific test method, test_foobar()

- Test can also be run on a specific class as well

2- Don't know why test need coverage? Run report

- Stuck on why test coverage is low and can't think of any place to increase it?

```
./bin/phpunit --coverage-html reports --config ./circleci/phpunit.xml
```

This generate a folder called `reports` with detail HTML reports on how covered a php script is; green line means covered / red line means its not. Goal is to convert as many red to green as possible.

Image1: Running the command and waiting for the report to generate in HTML format

```
yhou:onecms-universal-taxonomy yhou$ ./bin/phpunit --coverage-html reports --config ./circleci/phpunit.xml
Installing...
Running as single site... To run multisite, use -c tests/phpunit/multisite.xml
Not running ajax tests. To execute these, use --group ajax.
Not running ms-files tests. To execute these, use --group ms-files.
Not running external-http tests. To execute these, use --group external-http.
PHPUnit 6.5.13 by Sebastian Bergmann and contributors.

..... 43 / 43 (100%)

Time: 14.72 seconds, Memory: 38.00MB

OK (43 tests, 82 assertions)

Generating code coverage report in HTML format ... done
```

Image2: Viewing the folders

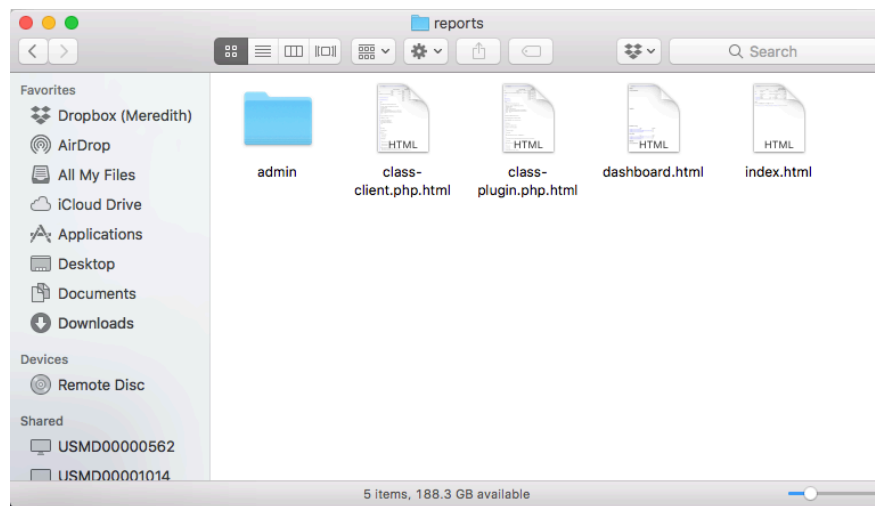


Image3: Viewing the report (Green indicate coverage while Red indicate missing coverage)

```

105  /**
106   * Returns array after searching the taxonomy tree in the cache .Cache will be valid for one
107   *
108   * @param string $search_string Search string.
109   * @return array|boolean
110   */
111  public static function fetch_taxonomy( $search_string = '' ) {
112      $tax_json = get_site_option( 'tax_taxonomy_tree', [] );
113      $tax_json_last_updated = get_site_option( 'tax_taxonomy_tree_last_updated', false );
114
115      if ( empty( $tax_json ) || empty( $tax_json_last_updated ) ) {
116          wp_schedule_single_event( time(), 'ceonix_universal_tax_json' );
117          $tax_json = 'ceonix_universal_taxonomy/UniversalTaxonomyTreeUniversalTaxonomyTree';
118          $date = ( time() - $tax_json_last_updated ) > 300 ? 1 : 0;
119          wp_schedule_single_event( time(), 'ceonix_universal_tax_json' );
120      }
121
122      if ( ! empty( $tax_json ) && is_array( $tax_json['norm'] ) ) {
123          $filtered_taxonomy = $tax_json['norm'];
124          if ( ! empty( $search_string ) ) {
125              $filtered_taxonomy = array_filter(
126                  $tax_json['norm'], function( $data ) use ( $search_string ) {
127                      $valid = strpos( $data['name'], $search_string ) !== false;
128                      if ( ! $valid && empty( $data['synonym'] ) && is_array( $data['synonym'] ) ) {
129                          $valid = strpos( $data['synonym'], $search_string ) !== false;
130                      }
131                      if ( $valid ) {
132                          break;
133                      }
134                  }
135              );
136          }
137          $valid = $valid || strpos( $data['definition'], $search_string ) !== false;
138          if ( $valid ) {
139              return $valid;
140          }
141      }
142      return false;
143  }

```

3- How to mocking dependency using Mockery?

Suppose I have a code that I need to Mock `fetch\_taxonomy` from App class

```

function get_search_result () {
    if (isset( $_POST['query'] ) ) {
        $results = App::fetch_taxonomy( $query );
        if ( ! empty ( $result ) ) {
            return $results;
        }
    }
    return null;
}

```

I can mock App::fetch_taxonomy using Mockery

```
env COMPOSER=composer-test.json composer require --dev mockery/mockery
```

```

$_POST['query'] = 'Search for food';
$appMock = \Mockery::mock('overload:OneCMS\Universal_Taxonomy\Admin\App');
$appMock
    ->shouldReceive('fetch_taxonomy')
    ->with($_POST['query'])
    ->andReturn(
        (object) [
            'id' => 123,
            'terms' => [
                (object) [
                    'id' => 'term1',
                    'name' => 'term1-name',
                    'synonyms' => ['term1-syn'],
                    'definition' => ['term1-def'],
                ],
                (object) [
                    'id' => 'term2',
                    'name' => 'term2-name',
                    'synonyms' => ['term2-syn'],
                    'definition' => ['term2-def'],
                ]
            ]
        ]
    );

```

Error with: Mockery\Exception\RuntimeException: Could not load mock <insert class name>, class already exists?

```

* @runInSeparateProcess
* @preserveGlobalState disabled
// Sometime this solve the issue ^

```

4- How to fake a https server for testing calls like `wp\_safe\_remote\_get`?

Suppose I have a code that I need to fake an api call to \$utx_api_url

```
function get_universal_taxonomy_tree () {
    $args = [
        'headers' => [
            'method' => 'GET',
            'Accept' => 'application/json',
            'x-ssst' => $utx_auth_token,
        ],
    ];

    $data = wp_safe_remote_get( $utx_api_url, $args );

    if ( ! is_wp_error( $data ) && 200 === $data['response']['code'] )
    {
        $utx_data = json_decode( $data['body'] );
    }

    return $utx_data;
}
```

We can use WP_HTTP_TESTCASE

```
env COMPOSER=composer-test.json composer require --dev jdgrimes/wp-http-
testcase
```

```
// Have test extend `WP_HTTP_TestCase` instead of `WP_UnitTestCase`
```

```
// Make a mock api function (OPTIONAL!)
protected function mock_server_response() {
    return [
        'response' => [
            'code' => 200
        ],
        'body' => json_encode('Hi this is the body of the request')
    ];
}
```

```
// In your test method call the mock api function
public function
test_get_universal_taxonomy_tree_where_utx_data_is_returned()
{
    $this->http_responder = [ $this, 'mock_server_response' ];

    $result = OneCMS\Universal_Taxonomy\Client::
get_universal_taxonomy_tree();

    $this->assertEquals('Hi this is the body of the request', $result);
}
```

5- How to get past wp_verify_nonce?

Suppose I have a code that I require wp_verify_nonce to get pass

```
public function get_api_data() {
    if ( ! (isset( $_POST['psacheck'] ) && wp_verify_nonce(
sanitize_key( $_POST['psacheck'] ), 'get_api_data' ) ) ) {
        wp_send_json( array( 'error' => __( 'Error :
Unauthorized action' ) ) );
        return;
    }

    // Connection verified
    // Do whatever left...
}
```

We can do something like...

```
// Make a validate nonce function (OPTIONAL!)
private function setValidNonce($action) {
    $uid = $this->factory->user->create();
    wp_set_current_user( $uid );

    return wp_create_nonce($action);
}
```

```
// In your code
public function test_get_api_data() {
    $_POST['psacheck'] = $nonce;
    $nonce = $this->setValidNonce('get_api_data');

    $app = new OneCMS\Universal_Taxonomy\Admin\App();
    $app->get_api_data();
}
```

6- How to unit test CMB2 stuff?

Suppose I have this cmb2 layout stuff

```
public static function utx_metabox() {
    $utx_metabox_section = new_cmb2_box(
        [
            'id'          => 'onecms-utx',
            'title'       => __( 'Meredith Taxonomy', 'cmb2' ),
            'object_types' => [ 'post', 'page', 'gallery', 'project',
'category-page' ],
            'context'     => 'normal',
            'priority'    => 'high',
            'show_names'  => true, // Show field names on the left.
        ]
    );

    $utx_metabox_section->add_field(
        [
            'id'          => 'utag_ids',
            'type'        => 'utag_search_ajax',
            'desc'        => __( '(Type Tag names or synonyms)', 'cmb2' ),
            'sortable'    => true,    // Allow selected items to be sortable
(default false).
        ]
    );

    $utx_metabox_section->add_field(
        [
            'name'        => __( 'CT Metadata', 'cmb2' ),
            'id'          => 'tempo_ct_metadata',
            'type'        => 'text',
            'attributes' => array(
                'readonly'      => 'readonly',
                'data-codeeditor' => wp_json_encode(
                    array(
                        'codemirror' => array(
                            'mode'    => 'css',
                            'readOnly' => 'nocursor',
                        ),
                    ),
                ),
            ),
        ]
    );
}
```

We can use the cmb2 package

```
env COMPOSER=composer-test.json composer require --dev cmb2/cmb2
```

```
// In the plugin file
```

```
// Handle CMB2 Conditional Loading (mostly used for testing).  
if ( file_exists( trailingslashit( dirname( __FILE__ ) ) . '/vendor/cmb2  
/cmb2/init.php' ) && ! class_exists( 'CMB2' ) ) {  
    require_once trailingslashit( dirname( __FILE__ ) ) . '/vendor/cmb2/cmb2  
/init.php';  
}
```

```

public function test_utx_metabox() {
    \OneCMS\Universal_Taxonomy\Admin\App::utx_metabox();

    $cmb2_properties = cmb2_get_metabox('onecms_utx')->properties;

    $this->assertArraySubset(
        [
            'id' => 'onecms_utx',
            'title' => 'Meredith Taxonomy',
            'object_types' => [ 'post', 'page', 'gallery',
'project', 'category-page' ],
            'context' => 'normal',
            'priority' => 'high',
            'show_names' => true
        ],
        $cmb2_properties
    )

    $this->assertEquals(
        [
            'utag_ids' => [
                'id' => 'utag_ids',
                'type' => 'utag_search_ajax',
                'desc' => '(Type Tag names or synonyms)',
                'sortable' => true
            ],
            'tempo_ct_metadata' => [
                'id' => 'tempo_ct_metadata',
                'name' => 'CT Metadata',
                'type' => 'text',
                'attributes' => [
                    'readonly' => 'readonly',
                    'data-codeeditor' => '{"codemirror":{"mode":"css","
readOnly":"nocursor"}}'
                ]
            ],
            $cmb2_properties['fields']
        );
    }
}

```

7- How to unit test wp_json_send / wp_die / die for rendering json?

Suppose I have this code that render json

```

public function render_my_json() {
    // ..... do something at the top

    die( json_encode( [ 'error' => __( 'Error : Unauthorized action' ]
);
    //      or wp_die( json_encode( [ 'error' => __( 'Error :
Unauthorized action' ] );
    return
}

```

First, don't use die() or wp_die(), use wp_send_json instead since its more proper.

IMPORTANT: DO NOT USE json_encode or wp_json_encode, that is done automatically via wp_send_json, if your code doesn't work without it then that means the frontend javascript code might not be correct.

If frontend javascript code contains `data = \$.parseJSON( data );` then remove that line because there is no need to convert data to JSON object since it is already a JSON object

```

public function render_my_json() {
    wp_send_json( [ 'error' => __( 'Error : Unauthorized action' ) ] );
}

```

Then we add the test

**This testing method is also useful for handling scenarios where a method exits early.

```

public function test_render_my_json() {
    add_filter( 'wp_doing_ajax', '__return_true' );
    add_filter( 'wp_die_ajax_handler', [ $this, 'get_die_handler' ] );

    ob_start();
    $cmb_field_utag_search_ajax = new
OneCMS\Universal_Taxonomy\Admin\CMB_Field_Utag_Search_Ajax();
    $cmb_field_utag_search_ajax->cmb_utag_search_ajax_get_term();
    $content = ob_get_contents();
    $this->assertEquals('{"error":"Error : Unauthorized action"}',
$content);
    ob_end_clean();
}

// PLEASE REMEMBER TO ANNOTATE IT IN YOUR CODE
public function get_die_handler() {
    return [ $this, 'die_handler' ];
}

// PLEASE REMEMBER TO ANNOTATE IT IN YOUR CODE
public function die_handler( $message ) {}

```

Alternatively,

Running a test on a function that ``exit`` or ``wp_die`` that throws an error such as:

```
// if we didn't redirect out, then we fail.
```

```
wp_die( esc_html( __( 'Invalid Post ID', 'onecms-revisions' ) ) );
```

Can be tricky to get coverage on. You can tell the test to expect an error and to not crash when that error is reached by using:

```
$this->expectException(WPDieException::class);
```

Reading on ``expectException``:

<https://github.com/ArtOfWP/wp-testing/blob/master/src/Exceptions/WPDieException.php>

<https://www.sitepoint.com/forums/showthread.php?501505-Simpletest-testing-exceptions>

Example test:

```
/**
```

- @covers OneCMS\Revisions\Controller::create

```
*/
```

```
public function test_create()
```

```
    $this->expectException(WPDieException::class);
```

8- How to mock built-in wordpress function? (sometime this works)

Certain wordpress function can be mocked in a unit test.

Suppose I have a function like this

```
// In a class call fooClass
public static function add_submit_box_article_variant( $post ) {
    if ( 'post' !== $post->post_type ) {
        return;
    }

    $name = '';
    $article_variant = get_the_terms( $post, 'article_variant' );
    if ( is_object( $article_variant[0] ) ) {
        $name = $article_variant[0]->name;
    }

    return $name;
}
```

And I want to test the case where name is not empty (that would mean `get_the_term` must return something that has a name!)

I can do

```

public function
test_add_submit_box_article_variant_will_return_no_empty_name {
    //      Apply the override
    add_filter('get_the_terms', function() {
        return (object) [
            'name' => 'bar'
        ];
    });

    $name = fooClass::add_submit_box_article_variant('post');

    //      $name will be 'bar' because I override it when I did the
    `add_filter`
}

```

9 - How do I test all these HTML echo / dump?

Given a code that has

```

In class renderHtml
public static function renderContainer($message) {
    echo "<div class=\"container\">{$message}</div>";
}

```

I can test it using

```

public function test_spit_html() {
    $message = 'This is my container yo!';
    ob_start();
    renderHtml::renderDiv($message);
    $buffer = ob_get_contents();
    ob_end_clean();

    $expected = '<div class="container">'.
                $message.
                '</div>';

    // Don't really need preg_replace if you don't want to make it
    prettier.
    $this->assertEquals(
        preg_replace( '/\s*/', '', $expected ),
        preg_replace( '/\s*/', '', $content )
    );
}

```

10- How to unit test filter_input?

Assume your giving a code that has filter_input

```
//      Class: app
public function validate_form() {
    $email = filter_input( INPUT_POST, 'nonce', FILTER_SANITIZE_STRING
);

    if ( ! empty( $email ) ) {
        //..... perform additional business logic work to process
email
        return true;
    }

    return false;
}
```

Its currently improbable to mock filter_input without installing extension.

Because of that, we need to make changes to the UI

(check this: https://codex.wordpress.org/Function_Reference/sanitize_key for some additional sanitization filter, don't let this link name trick you =>))

```
//      Class: app
public function process_email() {
    if ( isset ( $_POST['email'] ) ) {
        $email = sanitize_email( wp_unslash( $_POST['email'] ) );
        //..... perform additional business logic work to process
email
        return true;
    }

    return false;
}
```

Then we can test it by mocking

```
public function test_process_email() {
    $_POST['email'] = 'john.doe@email.com';

    $app      = new app();
    $result = $app->process_email();

    $this->assertTrue($result);
}
```

Given:

```
/**
 * Register all cmb2 boxes.
 */
public static function register() {
    add_action( 'cmb2_init', [ get_called_class(), 'body_box' ], 40 );
    add_filter(
        'tempo_send_post_types', function ( $post_types ) {
            $post_types[] = 'post';

            return $post_types;
        }
    );
}
```

You can test using this method:

```
/**
 * Test register
 *
 * @covers OneCMS_CMB2\V_1_0\Register_Fields::register
 *
 * @return void
 */
public function test_register() : void {
    $result = OneCMS_CMB2\V_1_0\Register_Fields::register();

    // Manually test filter tempo_send_post_types due to callback.
    $post_types_array = apply_filters('tempo_send_post_types', ['gallery']);

    $this->assertNotFalse(
        has_action(
            'cmb2_init', [
                'OneCMS_CMB2\V_1_0\Register_Fields',
                'body_box',
            ]
        )
    );

    $this->assertEquals(['gallery', 'post', 'post'], $post_types_array);

    $this->assertNull( $result );
}
```

Given

```
public static function add_submit_box_article_variant( $post ) {
    if ( 'post' !== $post->post_type ) {
        return;
    }

    $article_variant = get_the_terms( $post, 'article_variant' );

    if ( is_object( $article_variant[0] ) ) {
        return true;
    }

    return false;
}
```

We can test `get\_the\_terms` via factory terms

```
public function test_add_submit_box_article_variant() : void {
    $post = $this->factory->post->create_and_get();
    $this->factory->term->add_post_terms( $post->ID, ['something'],
    'article_variant');

    $result = OneCMS_CMB2\V_1_0\Register_Fields::
    add_submit_box_article_variant($post);

    $this->assertTrue( $result );
}
```

13 - How do I test wp-cli commands?

Some of the plugins might be having cli commands and that cannot be invoked as a normal callback. So we need to add wp-cli as a dependency and include the required files in the bootstrap file for test cases.

Add a dependency of wp-cli under "require-dev" in the composer-test.json

composer-test.json should like this after adding the wp-cli dependency

```
{
  "config": {
    "bin-dir": "bin",
    "vendor-dir": "vendor"
  },
  "require-dev": {
    "wp-coding-standards/wpcs": "^0.14",
    "phpunit/phpunit": "^6",
    "rregeer/phpunit-coverage-check": "^0.1.6",
    "cmb2/cmb2": "^2.5",
    "mockery/mockery": "^1.2",
    "jdgrimes/wp-http-testcase": "^1.3",
    "wp-cli/wp-cli": "2.1.0"
  },
  "scripts": {
    "test": " ./bin/phpcs --runtime-set ignore_warnings_on_exit 1 --
standard=./.circleci/codesniffer.ruleset.xml --extensions='php,js,css' ./",
    "fix": " ./bin/phpcbf --standard=./.circleci/codesniffer.ruleset.xml --
extensions='php,js,css' ./",
    "phpunit": " ./bin/phpunit --stop-on-error"
  }
}
```

Include the required files in the bootstrap for testcases. Add the below code to bootstrap.php

```
$vendorDir = dirname( dirname( __FILE__ ) ) . '/vendor';

require_once dirname( dirname( __FILE__ ) ) . '/tests/src/Logger.php';

if ( ! defined( 'WP_CLI_ROOT' ) ) {
    define( 'WP_CLI_ROOT', $vendorDir . '/wp-cli/wp-cli' );
}

include WP_CLI_ROOT . '/php/utils.php';
include WP_CLI_ROOT . '/php/dispatcher.php';
include WP_CLI_ROOT . '/php/class-wp-cli.php';
include WP_CLI_ROOT . '/php/class-wp-cli-command.php';

\WP_CLI\Utils\load_dependencies();

\WP_CLI::set_logger( new OneCMS\Test_WP_CLI\Logger() );

// Prepare the static runner object.
$state = new \WP_CLI\Bootstrap\BootstrapState();
$step_instance = new \WP_CLI\Bootstrap\ConfigureRunner();
$state          = $step_instance->process( $state );
```

Now we need to create a logger file which contains the logger class which is set in `\WP_CLI::set_logger( new OneCMS\Test_WP_CLI\Logger()`);

Create `Logger.php` file in the `tests/src/` and add the below code

```
<?php

namespace OneCMS\Test_WP_CLI;

/**
 * PHPUnit logger
 */
class Logger {

    /**
     * @param string $message Message to write.
     *
     * @return string
     */
    public function info( $message ) {
        print $message;
    }

    /**
     * @param $message
     *
     * @return mixed
     */
    public function success( $message ) {
        print $message;
    }

    /**
     * @param $message
     *
     * @return mixed
     */
    public function warning( $message ) {
        print $message;
    }

    /**
     * @param $message
     *
     * @return mixed
     */
    public function error( $message ) {
        print $message;
    }

    /**
     * @param $message_lines
     *
```

```

    * @return string
    */
    public function error_multi_line( $message_lines ) {
        $message = implode( "\n", $message_lines );

        print $message;
    }

    public function debug() {

    }
}

```

Now WP_CLI class should be available in your test cases.

Note: We need to invoke the function which is registering our commands before invoking the command

Example

suppose my command is onecms-srm find-outbound-domains

```

// We should make WP_CLI to true
if ( ! defined( 'WP_CLI' ) ) {
    define( 'WP_CLI', true );
}
// Register my commands.
$plugin = new OneCMS\Safe_Redirect_Manager\plugin();
$plugin->onload(new stdClass());

// Invoke my command

WP_CLI::run_command(array('onecms-srm', 'find-outbound-domains'));

$content = ob_get_contents();
ob_end_clean();

$content will be a string which is outputted by WP_CLI from the commands
WP_CLI::success , WP_CLI:Error etc.. (This is coming from logger)

You can do the assertion depending on the message from the WP_CLI command.

```

```

// Commands with arguments
WP_CLI::run_command(array('onecms-srm', 'add', 'from', 'to', 301, 100));

```

```
$assoc_args = array('log-meta' => 'hello');
WP_CLI::run_command(array('onecms-srm', 'send-to-content-router'),
$assoc_args, []);
```

You can also use WP_CLI::runcommand

```
WP_CLI::runcommand('onecms-srm find-outbound-domains');
```

WP_ERROR will terminate your execution as it contains exit(), so try to avoid entering a condition with WP_ERROR

14- How to unit test is_admin()?

Given

```
public function wrapper_for_is_admin_function(){
    if (is_admin()) {
        return true;
    }

    return false;
}
```

You can do

```
public function test_wrapper_for_is_admin_function_where_is_admin_is_true()
{
    set_current_screen('edit.php');

    $result = $this->wrapper_for_is_admin_function();

    $this->assertTrue($result);
}
```

15- How to unit test get_term?

Given

```
$term = get_term( $editorial_program_id, 'category' );
```

You can do

```
$term = $this->factory->term->create_and_get( [ 'taxonomy' => 'category',  
  'name' => 'category' ] );  
$post = $this->factory->post->create_and_get(  
  [  
    'meta_input' => [  
      'editorial_program_terms' => $term  
    ]  
  ]  
);
```